

Excellent questions. These are the kinds of things that help you understand Java deeply.

Question 1: If int has no class, why does int.class work?

You're right that:

```
int
```

is not a class.

It is a primitive type.

When you write:

```
int.class
```

Java does not magically turn int into a class.

Instead, Java provides a special metadata object for primitive types too.

Think of it like:

```
int.class
```

means:

"Give me information about the primitive type int."

Similarly:

```
double.class
```

```
boolean.class
```

```
char.class
```

all work.

So:

```
String.class
```

means:

Information about the String class.

While:

```
int.class
```

means:

Information about the primitive type int.

Both return a Class object, but one represents a class and the other represents a primitive type.

Question 2: Why can't int hold null?

Let's first understand what null means.

```
null
```

means:

No object exists.

Example:

```
String name = null;
```

Memory:

```
name  
↓  
nothing
```

No String object exists.

Why is this allowed?

Because String is a reference type.

A reference variable can either:

Point to an object

or

Point to nothing (null)

Example:

```
String name = "John";
```

Memory:

```
name  
↓  
"John"
```

Example:

```
String name = null;
```

Memory:

```
name  
↓  
nothing
```

Still valid.

Now Look At int

```
int age;
```

An int variable does not store a reference.

It stores the actual number directly.

Example:

```
int age = 25;
```

Memory:

```
age  
↓  
25
```

There is no object.

No reference.

Just the value.

Now imagine:

```
int age = null;
```

Java asks:

"What number is null?"

Is it:

```
0 ?
```

or

```
100 ?
```

or

```
-5 ?
```

No.

null is not a number.

So Java says:

Compilation Error

because a primitive variable must contain a valid primitive value.

Easy Rule

Primitive types store values directly:

```
int  
double  
char  
boolean
```

They cannot be null.

Reference types store references:

```
String  
Scanner  
EmailResponseDTO  
ArrayList  
Map
```

They can be null.

Then What If I Need an Integer That Can Be Null?

Java provides wrapper classes.

Primitive:

```
int
```

Wrapper class:

```
Integer
```

Example:

```
Integer age = null;
```

Valid.

Because:

```
Integer
```

is a class (reference type).

But:

```
int age = null;
```

Invalid.

Because:

```
int
```

is a primitive type.

Quick Comparison

```
int age = 25;
```

Stores:

```
25
```

directly.

```
Integer age = 25;
```

Stores a reference to an Integer object.

```
Integer age = null;
```

Valid.

```
int age = null;
```

Invalid.

Memory Picture

Primitive:

```
int age = 25;
```

```
age  
↓  
25
```

Reference:

```
String name = "John";
```

```
name  
↓  
Object
```

Reference with null:

```
String name = null;
```

```
name  
↓  
nothing
```

Allowed because references can point to "nothing."

Primitives cannot, because they are the value itself.

Then What If I Need an Integer That Can Be Null?

Java provides wrapper classes.

Primitive:

```
int
```

Wrapper class:

```
Integer
```

Example:

```
Integer age = null;
```

Valid.

Because:

```
Integer
```

is a class (reference type)

Explain what is wrapper class and what is this

This is one of the most important concepts in Java.

Problem With Primitive Types

Primitive types:

```
int  
double  
char  
boolean
```

store values directly.

Example:

```
int age = 25;
```

Memory:


```
age  
↓  
25
```

Because int is not an object:

```
int age = null;
```

is invalid.

Now suppose Java needs an object version of int.

Why?

Because many features in Java work only with objects.

Examples:

```
Collections  
Generics  
Maps  
Lists  
Frameworks
```

Example Problem

Suppose you create:

```
ArrayList<int> numbers =  
    new ArrayList<>();
```

Not allowed.

Why?

Because:

```
ArrayList
```

can only store objects.

So Java created a class called:

Integer

What Is Integer?

Think of it as:

Integer

=

Object version of int

This is called a Wrapper Class.

Because it "wraps" a primitive value inside an object.

Visual

Primitive:

```
int age = 25;
```

Memory:

25

Wrapper:

```
Integer age = 25;
```

Memory:

```
age
↓
Integer Object
  ↓
  25
```

The object is wrapping the value.

Hence the name:

```
Wrapper Class
```

Wrapper Classes For All Primitives

```
int      → Integer
double   → Double
char     → Character
boolean  → Boolean
long     → Long
float    → Float
byte     → Byte
short    → Short
```

Why Can Integer Be Null?

Because Integer is a class.

Example:

```
Integer age = null;
```

Memory:

```
age
↓
nothing
```

Valid.

Exactly the same as:

```
String name = null;
```

Because both are reference types.

Why Did Java Create Wrapper Classes?

Main reasons:

To Allow Null

Primitive:

```
int age = null;
```

✗ Invalid

Wrapper:

```
Integer age = null;
```

✓ Valid

To Use Collections

Not allowed:

```
ArrayList<int>
```

Allowed:

```
ArrayList<Integer>
```

Example:

```
ArrayList<Integer> marks =  
    new ArrayList<>();
```

Store values:

```
marks.add(90);  
  
marks.add(80);  
  
marks.add(70);
```

Utility Methods

Wrapper classes contain useful methods.

Example:

```
Integer.parseInt("123")
```

converts:

```
"123"
```

into:

```
123
```

What Actually Happens Here?

```
Integer age = 25;
```

Looks simple.

But internally Java does something like:

```
Integer age =  
    Integer.valueOf(25);
```

Java creates an Integer object.

Inside it stores:

```
25
```

Memory:

```
age
↓
Integer Object
↓
25
```

Real Life Analogy

Primitive:

```
25
```

is like a loose coin.

Wrapper:

```
Integer(25)
```

is like putting the coin inside a box.

The box is the object.

The coin is the primitive value.

Why String Doesn't Need A Wrapper?

Because:

```
String
```

is already a class.

Already an object.

Already a reference type.

So:

```
String name = null;
```

works naturally.

No wrapper needed.

Easy Interview Definition

Wrapper Class

A wrapper class is a class that provides an object representation of a primitive data type.

Examples:

```
int      → Integer
double   → Double
char     → Character
boolean  → Boolean
```

It allows primitive values to be used where objects are required, such as Collections, Generics, and Framework APIs.

What is object means here

This is where most beginners get confused because people say:

"Integer is an object."

But what exactly is an object?

Think of a class as a blueprint.

Example:

```
class Student {
}
```

This is just a design.

Like a house blueprint.

No real house exists yet.

When you create:

```
Student s = new Student();
```

Now a real thing is created in memory.

That real thing is called an:

Object

Real Life Example

Blueprint:

House Plan

Not a house.

Just a design.

Actual house built using that plan:

House

That's like an object.

In Java

Class:

Student

Blueprint.

Object:


```
new Student()
```

Actual Student object in memory.

Example

```
String name = "Hello";
```

People often don't realize this.

But:

```
"Hello"
```

is actually a String object.

Memory looks roughly like:

```
name
↓
String Object
↓
"Hello"
```

Now Back To Integer

When you write:

```
Integer age = 25;
```

Java creates an Integer object.

Think of it like:

```
age
↓
Integer Object
↓
25
```

The object contains the value.

That's why we say:

Integer

is the object version of:

int

Compare With int

```
int age = 25;
```

Memory:

```
age  
↓  
25
```

Just the value.

No object.

Compare With Integer

```
Integer age = 25;
```

Memory:

```
age  
↓  
Integer Object  
  ↓  
  25
```

Now an object exists.

Why Do We Care?

Objects can:

- Have methods
- Be stored in collections
- Be assigned null
- Be passed where Java expects an object

Example:

```
Integer age = 25;  
  
age.toString();
```

Works because age refers to an object.

But:

```
int age = 25;
```

is just a primitive value.

It is not an object.

One-Line Definition

An object is a real instance of a class that exists in memory.

Example:

```
String
```

= class

```
"Hello"
```

= String object

```
Integer
```

= class

25

inside an Integer wrapper = Integer object

Student

= class

new Student()

= Student object

That's what people mean when they say:

"Integer is an object" or "String is an object."

They mean there is an actual instance of that class sitting in memory.